Coding / Mps Acceleration

PDF edition of preserved repository document | docs/coding/mps_acceleration.md

Historical statements and test counts below retain their original scope. Consult the master report for current status.

Native MPS execution

The central device manager defaults to MPS and validates allocation before loading weights. Final neural evaluation refuses CPU placement and PYTORCH_ENABLE_MPS_FALLBACK=1. Explicit CPU reproduction is available for historical experiments; it is never an automatic substitute for the accelerated benchmark.

One resident backbone and tokenizer serve independent episodes through a dynamic request queue. Workers own their environment and memory; only the coordinator performs neural computation. Each episode's next request follows its actual previous tool result. Padding is left-sided to multiples of 64; contexts are not truncated. Model parameters, buffers, input tensors and LoRA computation reside on MPS. The train scripts explicitly place token tensors and model gradients on the selected device. Adam optimizer state follows parameters; scalar accounting and serialization remain host utilities.

The device audit found only explicitly placed training tensor constructors and the device-allocation probe. Adapter artifact validation opens CPU safetensor metadata without a forward pass. The retained CPU checkpoint backup is used to restore exact original values across precision trials, then released. Tokenization, decoding after inference, repository access, WASI execution, hidden tests, result serialization and Git remain CPU work.

The native runner tests float16/bfloat16 support and batches 1, 4, 8 and 16 within a ten-minute profiling cap. It compares fresh MPS episodes with preserved uncached CPU episodes from the identical checkpoint and prompts. Precision, padding and batching can affect greedy output; all observed correctness differences must be reported. Historical float32 learning cells remain separate from reduced-precision MPS cells. The speed comparison combines hardware, precision and batching; it is not an isolated GPU causal estimate.

Completed episodes are fsynced immediately and skipped on resume. The lockbox configuration is written before any locked outcomes. The original two-hour deadline is fixed, not reset by a process restart. Missing work remains explicitly not run.

The execution environment cannot allocate MPS despite native Terminal availability. The user launches the native command; tooling monitors workspace outputs and conducts CPU/WASI evidence checks. Native device placement and allocator measurements establish inference use; availability alone does not.

References: PyTorch MPS (https://docs.pytorch.org/docs/main/notes/mps.html), MPS fallback environment variable (https://docs.pytorch.org/docs/stable/mps_environment_variables.html).